

The "synapse" package

v0.1.0 MIT

Synapse is a typst package offering an easy way to have every reference of a concept linked to its introduction throughout a document. This is heavily inspired by the knowledge package for LaTeX, but with a typst-like API.

ROBOTECHNIC

 [@Robotechnic](#)

Table of Contents

[Index](#) 6

<code>#intro</code>	<code>#quote-rule</code>	<code>#synapse-config</code>
<code>#intro-ap</code>	<code>#syn</code>	
<code>#notion</code>	<code>#syn-wrapper</code>	

#synapse-config(`{mode}`): `"composition"`, (`{intro-style}`): `none`, (`{syn-style}`): `none`)

→ `none`

Change the synapse configuration.

Argument

(`mode`): `"composition"`

`str`

Change the synapse mode, which affects how notions are displayed. “paper” mode is optimized for print and will display notions in default document text colors (the color related styles will be ignored), “electronic” mode is optimized for screens and will display notions in color, and “composition” mode display everything like “electronic” mode but also highlight notions with a light red background if they are not introduced yet and display anchors with red markers. The default mode is “composition”.

Argument

(`intro-style`): `none`

`function`

A text function with any style arguments you want. This style will be applied to notions when they are introduced with the `intro()` function. The default `intro-style` is italic with a reddish fill color. Note that in “paper” mode, the fill and stroke styles will be ignored and set to none

Argument

(`syn-style`): `none`

`function`

A text function with any style arguments you want. This style will be applied to notions when they are used as synonyms with the `syn()` function. The default `syn-style` is bold with a blueish fill color. Note that in “paper” mode, the fill and stroke styles will be ignored and set to none

#notion(`{url}`): `none`, (`{style}`): `none`, `..{synonyms}`) → `content`

This function defines a notion, which is a concept that can be introduced and used as a synonym in the document. Each notion must have at least one synonym, which is the first positional argument. The notion can also have an optional URL and style. The URL makes the notion a link to an external resource, and the style allows you to customize how the notion is displayed when used as a synonym. adding a % in the label name allows to scope the notion so it will be displayed the same but will be considered a different notion. This can be useful when you want to use the same term with different meanings in the same document. For example, you could define `notion("set", "set%math")` to have two different notions for the word “set”, one for general use and one for mathematical use.

Argument

`{url}: none`

`str` | `none`

If provided, the notion will be a link to the provided URL. The default is none, which means the notion should have an internal definition in the document. If url is provided, the notion will not be able to be introduced with the `intro()` function.

Argument

`{style}: none`

`function` | `none`

A text function with any style arguments you want. The default is none, which means the notion will have the global default text style.

- synonyms: Any number of positional arguments can be provided as synonyms for the notion. Each synonym must be unique and cannot be used as a synonym for another notion.

#`intro(..(args))` → `content`

This function is used to introduce a notion for the first time in the document. It takes a notion and an optional body. If the notion is a string, it will be introduced as is. The introduced notion will be displayed with the `intro`-style defined in the synapse configuration.

If the provided notion is a function, the remaining function arguments will be passed to the function. See `syn-wrapper` for more details on how to use this with functions.

If the notion has already been introduced before or if the notion has a URL, an error will be thrown to prevent introducing the same notion multiple times or introducing a notion that is meant to be used as a link to an external resource.

Argument

`..(args)`

`str` | `content` | `function`

The text notion to introduce and either a body or the additional parameter to the introduced function (see `syn-wrapper()`).

#`intro-ap((notion))` → `content`

This function allows you to define an anchor point for a notion. It should be paired with an `intro()` call to work, else it won't have any effect. Once introduced, the notion will be linked here instead of the first introduction point. This is useful for instance when the notion is introduced in a paragraph but you want to link to the paragraph beginning instead of the notion itself. In composition mode, the anchor will be marked with a red marker to indicate that it is an anchor point for a notion. In paper and electronic modes, the anchor will not be visible but will still be linked to the notion introduction point.

- `notion (str)`: The notion to introduce. This should be the name of a notion defined with the `notion()` function.

#syn(..{args}) → **content**

This function is used to use a notion as a synonym in the document. It takes a notion as an argument. If the notion has been introduced before with the `intro()` function, it will link to the introduced notion. If the notion is not defined, in composition mode, it will be displayed with a highlight and a reddish fill to indicate that it is an undefined notion if in compose mode.

- `..args (str, content)`: The text notion to use as a synonym and an optional body. The body is used to change how the notion is displayed when used as a synonym.

#syn-wrapper({notion}, {function}) → **function**

This function is a wrapper for a notion that allows you to define how the notion should be displayed depending on the arguments passed. This function can also be used as an argument for the `intro()` function.

```
#notion("abs")

#let abs = syn-wrapper("abs", (wrapper, value) => {
  $wrapper(|)value#wrapper($|)$
})

#intro(abs, $x$) is a function that returns the absolute value of $x$.
For instance, $abs(-5)$ will return $5$ while $abs(42)$ will return the exact
same value of $42$.
```

abs
`|x|` is a function that returns the absolute value of x .
 For instance, `| - 5 |` will return 5 while `|42|` will return the exact same value of 42.

Argument

`{notion}`

str

The notion to wrap. This should be the name of a notion defined with the `notion()` function.

Argument

`{function}`

function

A function that takes at least a wrapper as argument and returns a content. The wrapper is a function that takes a content and applies either the `intro`-style or the `syn`-style to it depending on the context. The function can also take additional arguments, which will be passed when introduced or when used as a synonym.

#quote-rule({el})

Show rule to replace "<notion>" with #syn(notion) and "<notion>" with #intro(notion)

Part I

Index

I

[#intro](#) 2, 3
[#intro-ap](#) 2, 3

N

[#notion](#) 2

Q

[#quote-rule](#) 2, 4

S

[#syn](#) 2, 4
[#syn-wrapper](#) 2, 4
[#synapse-config](#) 2